

TESTES AUTOMATIZADOS EM PROJETO JAVA WEB

COM SPRING BOOT E JUNIT 5

ATIVIDADE AVALIATIVA — TESTES UNITÁRIOS CAIXA BRANCA

NOME:	TURNO:	TURMA:	DATA:
CURSO:	PROF.:		

1. Objetivo da Atividade

Desenvolver e executar testes unitários para um módulo de autenticação de usuários utilizando Java, Maven, JUnit 5 e IntelliJ IDEA, aplicando a técnica de teste caixa branca.

O aluno deverá analisar a estrutura interna dos métodos, identificar condições, decisões e caminhos de execução, elaborar casos de teste, implementar os testes automatizados e registrar os resultados.

2. Cenário do Projeto

A empresa SecureAccess Systems contratou sua equipe para desenvolver e testar o módulo de autenticação de um sistema corporativo. O módulo possui cadastro de usuário, validação de senha, autenticação, tratamento de exceções, controle de tentativas e níveis de acesso.

Nesta atividade, o aluno atuará simultaneamente como analista de testes e desenvolvedor de testes unitários, transformando os requisitos em casos de teste e verificando os diferentes caminhos internos do código.

3. Requisitos Funcionais

RF01 — Cadastro de Usuário

- O sistema deve permitir o cadastro de nome, login, senha e e-mail.
- Os campos obrigatórios não podem ser nulos ou vazios.

RF02 — Validação do Tamanho da Senha

- A senha deve possuir no mínimo 10 caracteres.
- A senha deve possuir no máximo 12 caracteres.

RF03 — Validação da Composição da Senha

- A senha deve possuir pelo menos uma letra.
- A senha deve possuir pelo menos um número.
- A senha deve possuir pelo menos um caractere especial.

RF04 — Validação de Campos Obrigatórios

- O sistema não deve permitir autenticação com usuário ou senha nulos ou vazios.

RF05 — Autenticação do Usuário

- Usuário existente e senha correta devem permitir o acesso.
- Usuário inexistente ou senha incorreta devem impedir o acesso.

RF06 — Tratamento de Exceções

- Tratar usuário inexistente, senha inválida, erro de autenticação, campos vazios e erro de conexão com o banco.

RF07 — Controle de Tentativas de Login

- Após 3 tentativas consecutivas de login inválido, a conta deve ser bloqueada.
- Usuário bloqueado não deve conseguir autenticar.

RF08 — Níveis de Usuário

- O sistema deve contemplar os níveis ADMIN, GERENTE e CLIENTE.

4. Requisitos Não Funcionais

- RNF01 — Utilizar Java, Maven, JUnit 5 e IntelliJ IDEA.
- RNF02 — Os testes unitários devem ser implementados com JUnit 5.
- RNF03 — Os testes devem ser independentes entre si sempre que possível.
- RNF04 — Cada teste deve possuir identificação clara do comportamento validado.
- RNF05 — O aluno deverá apresentar evidências da execução dos testes.

5. Técnica de Teste — Caixa Branca

Na técnica caixa branca, o testador conhece a estrutura interna do código. A análise deve considerar métodos, condicionais, condições compostas, retornos, exceções e caminhos de execução.

O objetivo é verificar se os diferentes caminhos relevantes foram exercitados pelos testes, e não somente verificar se uma entrada produz uma saída.

6. Código Base para Análise

```
public boolean validarSenha(String senha) {  
  
    if (senha == null || senha.isBlank()) {  
        return false;  
    }  
  
    if (senha.length() < 10 || senha.length() > 12) {  
        return false;  
    }  
  
    boolean possuiNumero =  
        senha.matches(".*\\d.*");  
  
    boolean possuiLetra =  
        senha.matches(".*[a-zA-Z].*");  
  
    boolean possuiEspecial =  
        senha.matches(".*[!@#%&*() ].*");  
  
    return possuiNumero &&  
        possuiLetra &&  
        possuiEspecial;  
}
```

7. Atividade 1 — Análise do Código

1. Quantas estruturas condicionais existem no método validarSenha()?

2. Quais condições fazem o método retornar false imediatamente?
3. Quais condições precisam ser verdadeiras para que o método retorne true?
4. Existem diferentes caminhos de execução? Justifique.
5. Por que testar somente uma senha válida não é suficiente para caracterizar a cobertura dos caminhos?

8. Atividade 2 — Plano de Teste Unitário

Elabore o plano de teste do método validarSenha(). Utilize os cenários abaixo como ponto de partida e complemente-os quando identificar caminhos não cobertos.

ID	Req.	Cenário	Entrada	Resultado esperado
CT01	RF02/RF03	Senha válida	Java@1234	Aceitar
CT02	RF02	Menor que 10 caracteres	9 caracteres	Rejeitar
CT03	RF02	Maior que 12 caracteres	13 caracteres	Rejeitar
CT04	RF03	Sem número	Java@Test	Rejeitar
CT05	RF03	Sem letra	123456@789	Rejeitar
CT06	RF03	Sem especial	Java123456	Rejeitar
CT07	RF04	Senha nula	null	Rejeitar
CT08	RF04	Senha vazia	""	Rejeitar
CT09	RF02	Exatamente 10 caracteres	10 caracteres	Aceitar se demais regras forem atendidas
CT10	RF02	Exatamente 12 caracteres	12 caracteres	Aceitar se demais regras forem atendidas

9. Atividade 3 — Cenários do Analista de Testes

CT11 — Senha com exatamente 10 caracteres

A senha está no limite mínimo. O sistema deve aceitar? Qual requisito e qual caminho são exercitados?

CT12 — Senha com exatamente 12 caracteres

A senha está no limite máximo. O sistema deve aceitar? Esse cenário é um teste de fronteira? Justifique.

CT13 — Senha com 9 caracteres

Qual condição será satisfeita? O restante das condições precisa ser executado?

CT14 — Letra e número, sem especial

Qual variável será false? Qual o resultado da expressão possuiNumero && possuiLetra && possuiEspecial?

CT15 — Número e especial, sem letra

Qual variável será false? O método deve aceitar ou rejeitar?

CT16 — Letra e especial, sem número

Qual regra de negócio está sendo violada?

10. Atividade 4 — Teste de Valores de Fronteira

Para a regra de senha entre 10 e 12 caracteres, crie e execute testes para 9, 10, 11, 12 e 13 caracteres.

- Quais são os limites da regra?
- Quais valores estão fora dos limites?
- Por que os valores imediatamente antes e depois do limite são importantes?
- Quais casos devem resultar em true e quais devem resultar em false?

11. Atividade 5 — Implementação com JUnit 5

Crie a classe `ValidacaoSenhaServiceTest` e implemente os casos de teste identificados no plano.

```
class ValidacaoSenhaServiceTest {  
  
    private ValidacaoSenhaService service =  
        new ValidacaoSenhaService();  
  
    @Test  
    void deveAceitarSenhaValida() {  
        String senha = "Java@1234";  
  
        boolean resultado =  
            service.validarSenha(senha);  
  
        assertTrue(resultado);  
    }  
}
```

12. Atividade 6 — Autenticação

ID	Cenário	Resultado esperado
AUT01	Usuário válido + senha válida	Autenticar
AUT02	Usuário inexistente	Rejeitar
AUT03	Senha inválida	Rejeitar
AUT04	Usuário nulo	Rejeitar
AUT05	Senha nula	Rejeitar
AUT06	Usuário vazio	Rejeitar
AUT07	Senha vazia	Rejeitar
AUT08	Usuário bloqueado	Rejeitar
AUT09	1ª tentativa inválida	Permanecer desbloqueado
AUT10	2ª tentativa inválida	Permanecer desbloqueado
AUT11	3ª tentativa inválida	Bloquear
AUT12	Tentativa após bloqueio	Rejeitar

13. Atividade 7 — Plano de Teste do Bloqueio

Cenário A — 1ª tentativa inválida

A conta deve ser bloqueada? Justifique.

Cenário B — 2ª tentativa inválida

A conta deve ser bloqueada? Justifique.

Cenário C — 3ª tentativa inválida

Qual deve ser o estado da conta?

Cenário D — Usuário já bloqueado

Se a senha estiver correta, o sistema deve permitir acesso? Justifique pelo RF07.

14. Atividade 8 — Perguntas do Analista de Testes

1. Qual requisito está sendo validado pelo teste?
2. Qual método está sendo testado?
3. Qual é a condição de entrada?
4. Qual é o resultado esperado?
5. Qual é o resultado obtido?

6. O teste foi aprovado ou reprovado?
7. Qual caminho interno do código foi executado?
8. Qual condição ou decisão foi coberta?
9. Existe alguma condição ainda não testada?
10. O caso é normal, exceção ou valor de fronteira?
11. Se o teste falhar, qual requisito poderá estar sendo violado?
12. O teste pode ser executado novamente de forma independente?

15. Modelo de Registro de Execução

ID do Teste	Preencher pelo aluno
Requisito	Preencher pelo aluno
Método	Preencher pelo aluno
Cenário/Entrada	Preencher pelo aluno
Resultado Esperado	Preencher pelo aluno
Resultado Obtido / Status / Observação	Preencher pelo aluno

16. Atividade 9 — Análise de Cobertura

- Todas as condições do método foram testadas?
- Foi testado o caminho de senha null?
- Foi testada senha vazia?
- Foram testados os limites inferior e superior?
- Foi testada a ausência de número, letra e caractere especial?
- Foi testada uma senha que atende a todas as condições?
- Existe algum caminho relevante ainda não coberto? Justifique.

17. Entrega do Aluno

- Código-fonte do projeto Java.
- Classe de serviço contendo as regras de negócio.
- Classe(s) de teste JUnit 5.
- Plano de teste com requisitos, cenários, entradas e resultados.
- Análise caixa branca com condições, decisões, caminhos e valores de fronteira.
- Evidências da execução dos testes no IntelliJ IDEA.

18. Critérios de Avaliação

Critério	Pontuação
Identificação e interpretação dos requisitos	2,0
Elaboração do plano de testes	2,0
Implementação dos testes com JUnit 5	2,0
Aplicação da técnica de caixa branca	2,0
Análise dos resultados e cobertura	2,0
TOTAL	10,0

19. Tecnologias Obrigatórias

- Java
- Maven
- JUnit 5
- IntelliJ IDEA
-
-

20. Desafio Final — Análise do Analista de Testes

O analista informa: “A equipe possui 10 testes automatizados e todos estão passando.”

Pergunta: Isso significa necessariamente que o módulo está completamente testado? Justifique considerando requisitos, caminhos de execução, condições verdadeiras e falsas, valores de fronteira, exceções e cobertura de testes.

A resposta deve ser técnica e fundamentada, evitando respostas baseadas somente na quantidade de testes executados.